

1. Definition : Event Handling in ReactJS is the process of handling user actions such as clicking a button, typing in an input box, submitting a form, or moving the mouse. React uses event handlers to perform specific actions when an event occurs.

2. Features of Event Handling in React

- Events are written in camelCase (e.g., onClick, onChange).
- Event handlers are passed as functions.
- React uses Synthetic Events which work the same across all browsers.
- We use curly braces {} to pass the function inside JSX.

3. Syntax

```
<button onClick={functionName}>Click Me</button>
```

4. Example Program

```
import React from "react";
function App() {
  function showMessage() {
    alert("Button Clicked!");
  }
  return (
    <div>
      <h2>React Event Handling Example</h2>
      <button onClick={showMessage}>Click Here</button>
    </div> );
}
```

```
export default App;
```

5. Explanation

- A function showMessage() is created.
- When the user clicks the button, the onClick event is triggered.
- The function runs and displays an alert message.

6. Common Events in React

- onClick – triggered when a button is clicked.
- onChange – triggered when input value changes.
- onSubmit – used when submitting forms.
- onMouseOver – triggered when the mouse moves over an element.

1. Definition : Binding Event Handlers in ReactJS means connecting an event handler function with the component so that the function can correctly access this keyword and component state/props.

In React class components, methods are not automatically bound to the component instance. Therefore, we must bind the event handler to use this.

2. Why Event Binding is Required

- **To access component state and props inside the event handler.**
- **To ensure the correct value of this refers to the component object.**
- **Without binding, this becomes undefined in the function.**

3. Method 1: Binding in Constructor

The event handler is bound inside the constructor using bind().

```
import React, { Component } from "react";
class App extends Component {
  constructor(props) {
    super(props);
    this.handleClick = this.handleClick.bind(this); }
  handleClick() {
    alert("Button Clicked"); }
  render() {
    return (
      <button onClick={this.handleClick}>Click</button> ); } }
export default App;
```

Explanation

- **this.handleClick = this.handleClick.bind(this);**
- **This binds the function with the component instance.**

4. Method 2: Using Arrow Function

Arrow functions automatically bind this.

```
<button onClick={() => this.handleClick()}>
  Click
</button>
```

5. Method 3: Class Property Arrow Function

```
handleClick = () => {
  alert("Button Clicked"); }
```

1. Conditional Rendering

Definition : Conditional Rendering in React means displaying different UI elements or components based on certain conditions. It works similar to if-else conditions in JavaScript.

Methods of Conditional Rendering

1. If-Else Statement
2. Ternary Operator
3. Logical AND (&&) Operator

Example

```
import React from "react";
function App() {
  const isLoggedIn = true;
  return (
    <div>
      {isLoggedIn ? <h2>Welcome User</h2> : <h2>Please Login</h2>}
    </div> ); }
export default App;
```

Explanation

- A variable `isLoggedIn` stores the condition.
- If the value is true, the message **Welcome User** is displayed.
- If the value is false, the message **Please Login** is shown.

Real-life Example

In an online shopping website, if a user is logged in, the system shows “Welcome User”; otherwise it shows “Login / Register”.

2. List Rendering

Definition : List Rendering in React is used to display multiple elements or components from an array of data. It is usually done using the `map()` function.

Syntax

```
array.map((item) => <element>{item}</element>)
```

Example

```
import React from "react";
function App() {
  const fruits = ["Apple", "Banana", "Mango"];
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li> ))}
    </ul> ); }
export default App;
```

Explanation

- An array `fruits` contains multiple values.
- The `map()` function loops through the array.
- Each item is displayed as a list element (`<li>`).
- `key` attribute helps React identify each element uniquely.

1. Lists in React

Definition : In React, lists are used to display multiple items or components from an array of data. React uses the JavaScript `map()` function to create lists of elements.

Syntax

```
array.map((item) => <element>{item}</element>)
```

Example

```
import React from "react";
function App() {
  const fruits = ["Apple", "Banana", "Mango"];
  return (
    <ul>
      {fruits.map((fruit) => (
        <li>{fruit}</li>  ))}
    </ul> ); }
export default App;
```

Explanation

- **An array `fruits` stores multiple values.**
- **The `map()` function iterates through each item.**
- **Each value is displayed as a list item (`<li>`) in the UI.**

2. Keys in React

Definition : A key is a special attribute used in React to identify each element uniquely in a list. It helps React efficiently update and manage the UI.

Syntax

```
<li key={uniqueValue}>{item}</li>
```

Example

```
import React from "react";
function App() {
  const fruits = ["Apple", "Banana", "Mango"];
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>  ))}
    </ul> ); }
export default App;
```

Explanation

- **`key={index}` assigns a unique identifier to each list item.**
- **React uses this key to track changes, updates, and deletions in the list.**

3. Importance of Keys

- **Helps React identify which items changed.**
- **Improves performance during UI updates.**
- **Prevents unnecessary re-rendering of elements.**

1. Definition : Using the array index as a key in React lists is called an anti-pattern because it can cause problems when the list items are added, removed, or reordered. React may incorrectly update the UI.

2. What is Index as Key

In list rendering, developers sometimes use the index value of an array as the key.

Example

```
import React from "react";
function App() {
  const fruits = ["Apple", "Banana", "Mango"];
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li> ))}
    </ul> ); }
export default App;
```

3. Problem with Using Index as Key

- When items are inserted or deleted, the index of items changes.
- React may reuse the wrong component.
- It can cause incorrect UI updates.
- It may affect performance and state management.

4. Example Problem

If a list is:

1. Apple
2. Banana
3. Mango

If Apple is removed, the index changes:

1. Banana
2. Mango

React may treat Banana as Apple, which can create UI errors.

5. Better Solution

Use a unique and stable key such as:

- Database ID
- Unique product ID
- Unique user ID

Example:

```
<li key={fruit.id}>{fruit.name}</li>
```

1. Definition

Form Handling in ReactJS refers to the process of collecting, managing, and processing user input from form elements such as text fields, checkboxes, radio buttons, and buttons. React handles forms using component state to store and update the input values.

2. Common Form Elements

The common HTML form elements used in React are:

- **<input>** – for text input
- **<textarea>** – for multiline text
- **<select>** – for dropdown lists
- **<button>** – to submit the form

3. Basic Steps in Form Handling

1. **Create form elements in JSX.**
2. **Use state to store input values.**
3. **Use event handlers like onChange and onSubmit.**
4. **Update the state when the user enters data.**
5. **Process the data when the form is submitted.**

4. Example Program

```
import React, { useState } from "react";
function App() {
  const [name, setName] = useState("");
  const handleChange = (event) => {
    setName(event.target.value); };
  const handleSubmit = (event) => {
    event.preventDefault();
    alert("Name submitted: " + name); };
  return (
    <form onSubmit={handleSubmit}>
      <label>Name:</label>
      <input type="text" value={name} onChange={handleChange} />
      <button type="submit">Submit</button>
    </form> ); }
export default App;
```

5. Explanation

- **useState** stores the input value.
- **onChange** updates the state when the user types.
- **onSubmit** handles the form submission.
- **event.preventDefault()** prevents page refresh.

1. Definition

Component Lifecycle Methods are special methods in React class components that are automatically called at different stages of a component's life. These stages include creation, updating, and removal of a component.

2. Phases of Component Lifecycle

React component lifecycle has three main phases:

1. Mounting
2. Updating
3. Unmounting

3. Mounting Phase

This phase occurs when the component is created and inserted into the DOM.

Important methods:

- **constructor()** – Initializes state and binds methods.
- **render()** – Returns the JSX to display UI.
- **componentDidMount()** – Executes after the component is added to the DOM.

Example use: Fetching data from a server.

4. Updating Phase

This phase occurs when state or props change, causing the component to update.

Important methods:

- **shouldComponentUpdate()** – Determines whether the component should update or not.
- **render()** – Re-renders the UI.
- **componentDidUpdate()** – Executes after the component is updated.

Example use: Updating UI when user data changes.

5. Unmounting Phase

This phase occurs when the component is removed from the DOM.

Important method:

- **componentWillUnmount()** – Used to clean up resources like timers or network requests.

Example use: Stopping background processes.

6. Lifecycle Diagram (Text Form)

Mounting Phase

constructor() → **render()** → **componentDidMount()**

Updating Phase

shouldComponentUpdate() → **render()** → **componentDidUpdate()**

Unmounting Phase

componentWillUnmount()

7. Real-life Example

In a weather application, **componentDidMount()** is used to fetch weather data from an API when the component loads.

1. Definition

Mounting Lifecycle Methods are the methods that are called when a React component is created and inserted into the DOM for the first time.

These methods help in initializing the component, rendering the UI, and performing tasks like data fetching.

2. Main Mounting Lifecycle Methods

1. constructor()

- It is the first method called in a component.
- Used to initialize state and bind event handlers.

2. static getDerivedStateFromProps()

- Called before rendering.
- Used to update state based on props.

3. render()

- Responsible for displaying the UI.
- Returns JSX elements to be rendered on the screen.

4. componentDidMount()

- Called after the component is successfully added to the DOM.
- Commonly used for API calls, data fetching, or starting timers.

3. Mounting Lifecycle Flow (Diagram)

constructor()



getDerivedStateFromProps()



render()



componentDidMount()

4. Example Program

```
import React, { Component } from "react";
class App extends Component {
  componentDidMount() {
    console.log("Component Mounted Successfully"); }
  render() {
    return <h2>Hello React</h2>; } }
export default App;
```

5. Explanation

- When the component loads, **render()** displays the UI.
- After the component appears in the browser, **componentDidMount()** runs and performs additional tasks.

1. Definition

Updating Lifecycle Methods are the methods that are called when a React component is updated due to changes in state or props. These methods control how the component re-renders and updates the UI.

2. Causes of Updating

A component updates when:

- **State changes**
 - **Props change**
 - **forceUpdate() is called**
-

3. Main Updating Lifecycle Methods

1. static `getDerivedStateFromProps()`

- **Called before rendering when props or state change.**
- **Used to update state based on new props.**

2. `shouldComponentUpdate()`

- **Determines whether the component should re-render or not.**
- **Helps improve performance by avoiding unnecessary updates.**

3. `render()`

- **Re-renders the component UI with updated data.**

4. `getSnapshotBeforeUpdate()`

- **Called before the DOM is updated.**
- **Captures information from the DOM before changes.**

5. `componentDidUpdate()`

- **Called after the component is updated in the DOM.**
 - **Used for operations such as network requests or DOM updates.**
-

4. Updating Lifecycle Flow (Diagram)

`getDerivedStateFromProps()`



`shouldComponentUpdate()`



`render()`



`getSnapshotBeforeUpdate()`



`componentDidUpdate()`

5. Example Program

```
import React, { Component } from "react";
class App extends Component {
  componentDidUpdate() {
    console.log("Component Updated");
  }
  render() {
    return <h2>React Updating Lifecycle Example</h2>;
  }
}
export default App;
```

1. Definition

A Pure Component in React is a special type of component that automatically performs a shallow comparison of props and state to decide whether the component should re-render or not.

If the props and state do not change, the component does not re-render, which improves performance.

2. Features of Pure Components

- Extends `React.PureComponent` instead of `Component`.
 - Performs automatic shallow comparison of props and state.
 - Prevents unnecessary re-rendering.
 - Improves application performance.
-

3. Syntax

`class ComponentName extends React.PureComponent`

4. Example Program

```
import React, { PureComponent } from "react";
class Example extends PureComponent {
  render() {
    return (
      <div>
        <h2>Pure Component Example</h2>
      </div> ); } }
export default Example;
```

5. Explanation

- The component extends `PureComponent`.
- React automatically checks if props or state have changed.
- If there is no change, React skips re-rendering the component.

6. Advantages

- Improves performance of React applications.
- Reduces unnecessary rendering.
- Makes applications faster and more efficient.

7. Real-life Example

In an e-commerce website, product lists may contain many components. Using Pure Components prevents unnecessary updates when the product data has not changed.

1. Definition

Fragments in React are used to group multiple elements together without adding an extra DOM node. They allow components to return multiple elements without wrapping them in a <div>.

2. Need for Fragments

- **Avoids unnecessary HTML elements in the DOM.**
- **Keeps the DOM structure clean.**
- **Improves performance and readability of code.**

3. Syntax of Fragments

Using `React.Fragment`

```
<React.Fragment>
  <h1>Hello</h1>
  <p>Welcome to React</p>
</React.Fragment>
```

Using Short Syntax

```
<>
  <h1>Hello</h1>
  <p>Welcome to React</p>
</>
```

4. Example Program

```
import React from "react";
function App() {
  return (
    <>
      <h2>React Fragment Example</h2>
      <p>This is a paragraph</p>
    </> ); }
export default App;
```

5. Explanation

- **The fragment <> </> groups multiple elements.**
- **React renders both elements without creating an extra <div> in the DOM.**

6. Real-life Example

In a table structure, fragments are useful when returning multiple <td> elements without adding extra tags that could break the table layout.